Homework 4 - Report

Álvaro Martínez Alfaro
Autonomous Robotics

April 26, 2026

1 Introduction

This report presents the development and evaluation of a reactive navigation system for a simulated mobile robot in the **Flatland** environment under **ROS2**. Flatland is a lightweight 2D simulatoir that enables rapid prototyping of robotic algorithms without the overhead of 3D physics engines.

The scenario consists of a robot equipped with a 360° LiDAR sensor navigating a 10×6m bounded arena in which four goal objects are placed at fixed positions. The robot's baseline behavior follows a simple reactive loop: move forward until an obstacle is detected, then turn in a random direction. This work extends the baseline through four progressive steps: (i) implementing a **goal detector** that uses laser readings to identify and remove goals upon close proximity, (ii) an **improved navigation** strategy that selects the direction with the most free space instead of turning randomly, (iii) systematic **experimentation** across different navigation modes, linear speeds, and laser ranges, and (iv) the design and evaluation of **custom maps** to assess the robustness of the developed behaviors.

The remainder of the report is structured as follows: Section 2 details the implementation of each component, Section 3 presents the experimental results, and Section 4 draws conclusions from the work.

2 Implementation details

The entire implementation is contained in two files: `__init__.py`, which defines the **Controller** ROS2 node responsible for robot perception, navigation, goal detection and experiment management; and `homework4.launch.py`, which exposes configurable arguments for the map type and output `csv` file.

2.1 Goal detector

Goal detection runs on every laser callback via `process_obstacle_detection`, which iterates over all remaining active obstacles and calls `should_remove_obstacle` for each one. This function applies a two-condition check:

1. **Proximity filter**: computes the Euclidean distance from the robot pose to the obstacle's known position. If it exceeds the `obstacle_detection_distance` threshold (1.0 m), the check is skipped immediately.

2. **Laser confirmation**: converts the vector to the obstacle into a robot-relative angle (`global_angle - robot_theta`, normalized to $[-\pi, \pi]$), maps the angle to a laser scan index via `angle_to_scan_index`, then reads the average laser range at that index using `get_mean_range` (a windowed average over $\pm$ `mean_window`) neighboring rays). If the difference between the laser reading and the computed distance is within the `obstacle_match_tolerance` (0.2 m), the obstacle is confirmed and removed from the active list.

When a goal is confirmed, `remove_obstacle` relocates it off-map via Flatland's `MoveModel` service, records the elapsed time since run start (for experimentation purposes), and removes it from the active set. When the last obstacle is removed, the run ends.

Here, the key parameters are the `obstacle_detection_distance`, which limits the search to a radius around the robot and is set to 1.0m; the `obstacle_match_tolerance`, which accounts for sensor noise and map inaccuracies and is set to 0.2m; and the `mean_window`, which smooths laser readings to improve robustness and its values vary per experiment (2 or 4).

2.2 Improved navigation

The navigation decision is made inside `lidar_callback`. On each scan, the averaged front laser reading (index $\lfloor N/2 \rfloor$, window = `mean_window`) is compared against `min_distance` (0.8m). If the path ahead is blocked, the controller triggers a rotation, either random or heuristic depending on the current `nav_mode`.

In the **random** mode (`start_random_rotation`), the robot simply picks a random angular speed uniformly in $[-\pi, \pi]$ rad/s. In the **improved** mode (`start_heuristic_rotation`), the robot scans all N laser rays and finds the index with the highest windowed average range (window = `mean_window`). It derives the corresponding robot-relative angle and turns at ± 1.5 rad/s, positive (left) if the angle is ≥ 0 and negative (right) otherwise.

In both modes, `rotation_iterations_left` is set to `random_rotation_iterations` (10) and counted down each callback tick, during which the robot rotates in place.

Additionally, a stuck detection mechanism was added to handle cases where the robot might get trapped in a corner or against a wall. Every 5 seconds, the controller checks

(a) Rect map (baseline) (b) Line map (c) TP map

Figure 1: The three maps used in the experiments.

whether the robot has moved at least 0.4 m. If not, it triggers a two-phase escape: first, it reverses for 20 callback ticks, and then it performs a random spin for 35 ticks before resuming normal navigation.

Finally, the key parameters here are the `min_distance` threshold for obstacle avoidance, which is set to 0.8m; the `random_rotation_iterations`, which determines how long the robot rotates, set to 10 iterations; and the turn speed fixed at 1.5 rad/s in the improved mode.

2.3 Map generation

Maps in Flatland are defined by grayscale *PNG* images paired with a *YAML* configuration file. Black pixels (0) are treated as walls, while white pixels (255) represent free space. The *YAML* sets the map resolution (0.05 m/pixel) and the origin $(-5.0, -3.0)$, so the 200×120 pixel image maps to a 10×6 m world with $(0, 0)$ at the center.

The script `image_generator.py` uses the *PIL* library to programmatically draw three maps:

- **Rectangular map**: only the four outer walls are present, creating a simple open arena for baseline testing.
- **Line map**: adds a thin vertical wall near the center (6 pixels wide $\times$ 40 pixels tall $= 0.3 \times 2.0$ m). It creates a partial barrier that splits the arena symmetrically.
- **TP map**: two offset rectangular pillars placed diagonally across the arena.

The three maps are shown in Figure 1. Both custom maps are designed to avoid the four goal positions $(\pm 2.5, \pm 1.5)$, so goals remain reachable in all configurations. Each map has its own `world_*.yaml` and `*_map.yaml` and can be selected at launch via the `world_name` argument.

Nav mode	Speed (m/s)	Window	Mean time (s)
Basic	0.5	2	80.71
	0.5	4	186.86
	1.0	2	34.70
	1.0	4	84.59
	2.0	2	12.39
	2.0	4	12.39
Improved	0.5	2	81.93
	0.5	4	191.63
	1.0	2	32.98
	1.0	4	78.14
	2.0	2	12.39
	2.0	4	12.39

Table 1: Mean completion time per configuration — Rect Map.

Nav mode	Speed (m/s)	Window	Mean time (s)
Basic	0.5	2	153.05
	0.5	4	171.41
	1.0	2	47.34
	1.0	4	67.09
	2.0	2	12.39
	2.0	4	12.49
Improved	0.5	2	151.85
	0.5	4	197.49
	1.0	2	62.48
	1.0	4	73.88
	2.0	2	12.39
	2.0	4	12.49

Table 2: Mean completion time per configuration — Line Map.

3 Experimentation

The experiments were conducted across **12 configurations**, combining 2 navigation modes (basic and improved), 3 linear speeds (0.5, 1.0 and 2.0 m/s), and 2 laser window sizes (2 and 4), each repeated **10 times per map**. The same set of configurations was tested on three maps: the baseline rect map, and the two custom maps (line and tp). The metric recorded is the total time to detect and remove all four goals, defined as the elapsed time from run start to the last goal removal. Between runs, the robot and all goals are automatically reset to their initial positions. The results are summarized in Tables 1, 2 and 3.

Nav mode	Speed (m/s)	Window	Mean time (s)
Basic	0.5	2	146.41
	0.5	4	297.42
	1.0	2	117.15
	1.0	4	165.63
	2.0	2	85.30
	2.0	4	72.76
Improved	0.5	2	121.57
	0.5	4	190.09
	1.0	2	70.72
	1.0	4	106.45
	2.0	2	45.54
	2.0	4	54.29

Table 3: Mean completion time per configuration — TP Map.

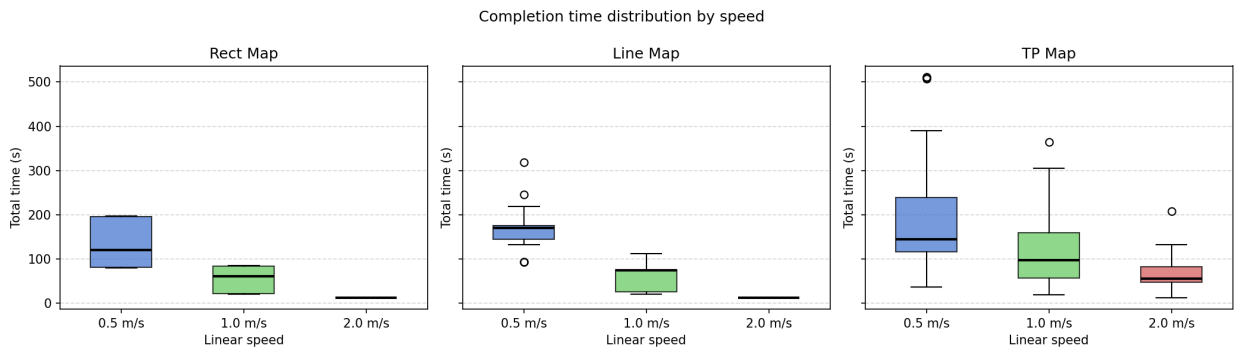

Figure 2: Completion time distribution by linear speed across the three maps.

3.1 Effect of linear speed

Linear speed is by far the dominant factor across all configurations and maps. At 2.0 m/s the robot detects all goals in less than 100 seconds consistently, while at 0.5 m/s it takes between 80 and 250 seconds, depending on the map. The boxplot in Figure 2 confirms that variance at 2.0 m/s is essentially zero or very low, which might be due to the fact that the robot follows the same deterministic path every run, while at 0.5 m/s it is very high, especially on the tp map, maybe because the robot has to do more obstacle avoidance maneuvers, following different paths each time. At 1.0 m/s results are intermediate but with notable variance depending on the window size, as discussed in Section 3.3.

3.2 Effect of navigation mode

The improved navigation mode, which selects the direction with the most free space instead of turning randomly, shows surprisingly **little improvement** over the basic mode. As shown in Figure 3, at 2.0 m/s both modes produce identical results across all maps; the robot is

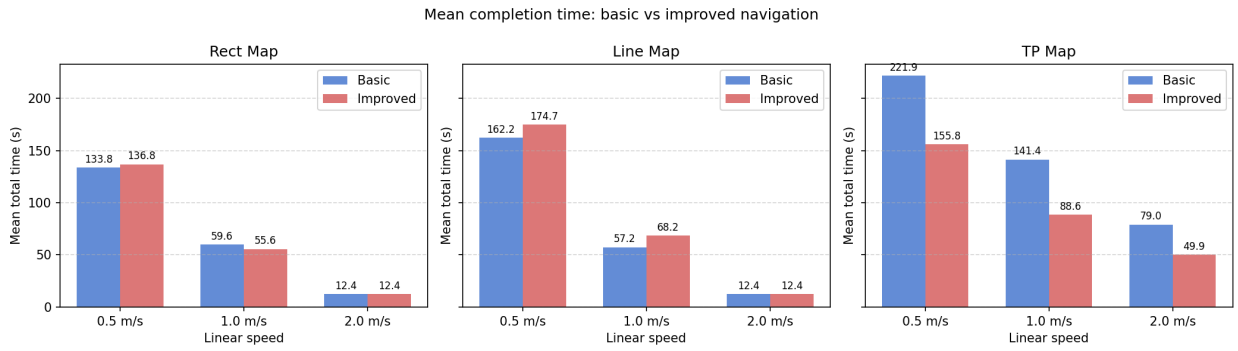

Figure 3: Mean completion time: basic vs improved navigation mode.

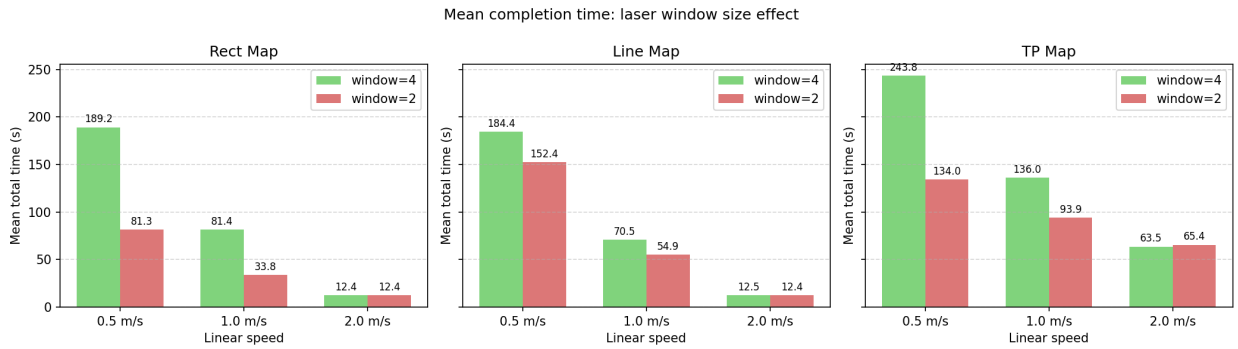

Figure 4: Mean completion time: effect of laser window size.

fast enough to reach all goals via the same deterministic path regardless of how it turns. At lower speeds, the differences are small and inconsistent; in some configurations the improved mode is marginally better, while in others the random mode performs equally well or even slightly better. This suggests that in relatively simple environments, like an open arena with few obstacles, most directions offer *similar free space* when an obstacle is encountered, giving the heuristic *little to exploit* over a random choice.

3.3 Effect of laser window size

The laser window size has a more notable effect than the navigation mode, particularly at 1.0 m/s. As shown in Figure 4, with window = 4 at 1.0 m/s the robot takes around 84 seconds consistently across all 10 runs on the rect map, with essentially zero variance. With window = 2 at the same speed, completion times drop to a 20–60 second range with much higher variance. This is a counterintuitive result, as one would expect that a larger averaging window should reduce sensor noise, leading to more reliable obstacle detection and better navigation decisions. Instead, it causes the robot to follow a path that happens to be suboptimal. At 2.0 m/s the window has no effect since the robot is fast enough to always follow the same path, while at 0.5 m/s both windows produce similarly high and variable completion times.

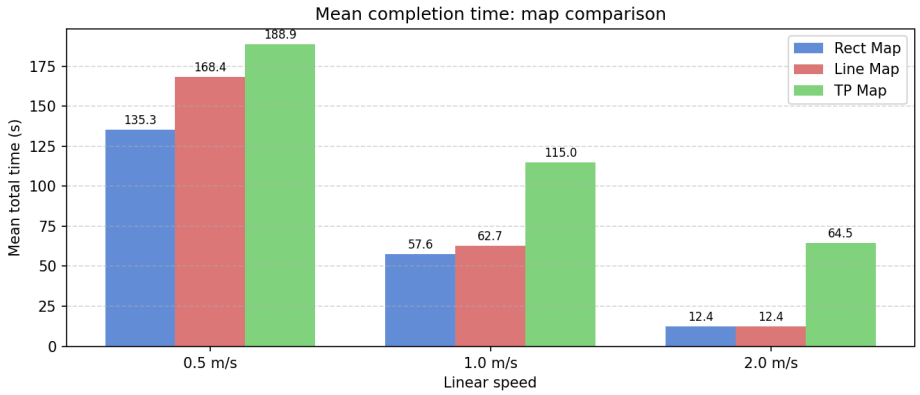

Figure 5: Mean completion time across maps by linear speed.

3.4 Map comparison

At 2.0 m/s the rect and line maps show identical completion times, while the tp map is slightly higher, reaching around a minute on average. This is expected since the tp map has two obstacles that may force the robot to change its path more often, while the line map has only one thin wall that can be easily circumvented. At lower speeds, the maps diverge more significantly. The line and tp maps introduce additional variance compared to the rect map, as the robot must navigate around the internal obstacles, following different paths each run. The tp map is the hardest at 0.5 m/s, producing the highest and most variable completion times across all configurations.

Overall, the map complexity has a secondary effect compared to linear speed, but it amplifies the variance already observed at lower speeds. Figure 5 summarises the mean completion time per map and speed.

4 Conclusion

This report presented the development and evaluation of a reactive navigation system for a simulated mobile robot in the Flatland environment under ROS2. The system was built incrementally, starting froim a baseline random-turn controller, a laser-based goal detector was added, the navigation strategy was improved with a free-space heuristic, and the whole system was tested across multiple configurations and three maps of increasing complexity.

The experiments revealed that linear speed is the dominant factor in performance, with the robot completing all goals in less than a minute at 2.0 m/s regardless of the map or navigation mode. The improved navigation mode showed little to no advantage over the basic one, which can be attributed to the relative openness of the arenas tested, where most directions offer similar free space when an obstacle is encountered, leaving the heuristic little to exploit. A more unexpected finding was the deterministic loop caused by the larger laser window size at 1.0 m/s, where the robot consistently followed the same suboptimal path, while the smaller window introduced enough variability to allow for faster completion times

in some runs. The tp map proved to be the most challenging scenario, particularly for the basic mode at low speeds.

As a final remark, there is one aspect that could be further explored or improved with additional time, that is testing on more complex maps, such as narrow corridors or maze-like layouts, would create situations where the free-space heuristic has a real advantage over random turning, allowing a fairer comparison between the two navigation modes.